

DevOps in Regulated Software Development: Case Medical Devices

Teemu Laukkarinen
Tampere University of Technology
Tampere, Finland
teemu.laukkarinen@tut.fi

Kati Kuusinen
University of Central Lancashire
Preston, UK
KKuusinen@uclan.ac.uk

Tommi Mikkonen
University of Helsinki
Helsinki, Finland
tommi.mikkonen@cs.helsinki.fi

Abstract—DevOps and continuous development are getting popular in the software industry. Adopting these modern approaches in regulatory environments, such as medical device software, is not straightforward because of the demand for regulatory compliance. While DevOps relies on continuous deployment and integration, regulated environments require strict audits and approvals before releases. Therefore, the use of modern development approaches in regulatory environments is rare, as is the research on the topic. However, as software is more and more predominant in medical devices, modern software development approaches become attractive. This paper discusses the fit of DevOps for regulated medical device software development. We examine two related standards, IEC 62304 and IEC 82304-1, for obstacles and benefits of using DevOps for medical device software development. We found these standards to set obstacles for continuous delivery and integration. Respectively, development tools can help fulfilling the requirements of traceability and documentation of these standards.

Keywords—Regulated software, agile development, DevOps, medical software development standards

I. INTRODUCTION

Ever since the invention of the compiler, software products have constantly become bigger and more complex systems. To tackle complexity issues in the development, software industry and scientific research have produced methods for executing development projects more efficiently. Approaches such as Scrum [1], Test-Driven Development [2], and Extreme Programming [3] have been widely adopted with successful outcomes – more reliable software, faster schedule, and cheaper delivery than with traditional methods.

Numerous industrial fields – including for instance automotive, space, and medical devices – require reliability, visibility, and traceability of the software project to ensure high quality, safety, and trustworthiness of the software. Many such industries have regulation authorities, which require proofs from manufacturers that the software running in their products has been developed in accordance to prescribed methods and tools before accepting the product to the markets. For instance, Food and Drug Administration (FDA) controls marketing and usage of medical devices in the United States, which must comply to FDA's requirements.

Standardization organizations, such as International Organization for Standardization (ISO), have created standards that describe the processes and the requirements for developing software in such manner that regulatory authorities will accept the product to the markets. By proofing compliance with these standards, a company can show to the authorities

that the software has been developed with a such process that ensures safety and trustworthiness. Due to these requirements of regulatory authorities, the development of medical devices have typically suffered from long development life-cycles.

Despite the general belief, regulatory standards for medical devices do not impose any particular software process model. There is evidence that agile and lean methods can be tailored for regulated environments by enriching them with more planning based practices [4]. However, there is not yet evidence how DevOps approaches [5] – practice where development and operations work together to rapidly deliver value to end users – fit to these regulatory standards.

In this paper we study the relation of the DevOps approach and regulated software development of medical devices. We pinpoint and analyze key clauses of IEC 62304 [6] and IEC 82304 [7] standards that affect the use of DevOps approaches. As a result, we found requirement for further development of DevOps approaches to enable better fit with the standards of medical device software development.

II. BACKGROUND AND MOTIVATION

For well over a decade, a constant trend in software companies has been the quest for faster development and deployment cycle. In the spirit of lean software development [8], parts of the development and deployment chain that do not add value are swiftly eliminated. For instance, there is little need for releases that integrate various features developed over several months into a single upgrade. Instead, whenever a feature is ready for delivery, it can be released immediately – given that a suitable development and deployment infrastructure is in place.

This practice, called continuous deployment [9], is rapidly becoming commonplace in software industry. Since there are several valid setups for processes and infrastructure to enable continuous deployment, and requirements vary concerning the development, delivery, and use of software, there is no one-size-fits-all solution for continuous deployment. Instead, different projects and organizations assume models that are best suited for their needs. While continuous deployment as an approach has clear goals in the big picture of software development, understanding the subtle differences in different setups in practice requires special attention.

Continuous deployment is often associated with DevOps, a term that has ambiguity in software industry and can mean anything that combines software development, quality assurance, and/or deployment operations to close the gap

between developers and operators [10]. Therefore, we selected definition of DevOps for this paper as development processes that reduce repetitive tasks in development, quality assurance, and deployment with help of automation tools and work-flows. It improves speed of the delivery, and collection of the user feedback through continuous operation. It reduces development costs, setup time, and failure recovery time through automation.

Continuous integration – the practice of constantly merging working copies to a shared mainline – and continuous deployment are considered key parts of DevOps. Continuous integration process typically includes automated build, unit tests, and integration tests for modified source code that is automatically pulled from a revision control system. Continuous deployment typically deploys the new build into production after successful continuous integration pass, but the deployment can also consist of such automated tasks as instancing cloud computing platform, deploying virtual machines, and installing and configuring operating system, supporting software, and libraries. With applications of varying end user loads, continuous deployment can automatically scale up and down the deployment on cloud computing platforms depending on the demand.

As both DevOps and continuous deployment aim at constantly delivering added value to end users, automation is needed to reduce repetitive work. This repetitive work in particular includes quality assurance, calling for automated tests, checks on code complexity, and so on. In connection with regulated development, automating such repetitive tasks is not straightforward, as the applicable standards typically rely on manual checks, validation, auditions, and additional approvals that cannot be fully automated. In fact, a common claim among engineers is that regulated software development needs waterfallish development [11], and no agile approach is applicable. However, this claim is questionable when agile approaches are tailored to fit regulation needs [4].

III. RELATED WORK

We could not identify related research on DevOps for medical device software development, and research on DevOps in embedded software domain is also scarce [12]. Medical device software is essentially embedded software although pure software medical devices exist. Lwakatare et al. [12] have listed numerous challenges in adopting DevOps in embedded software development: hardware dependency, limited visibility to customer environment, lack of deployment technology for customer specific environments, and incapability to get usage data from customer environment.

Ebert et al. [13] add that continuous deployment to the customer environment is not a feasible goal for safety-critical software, because failure is not an option, and rapid customer feedback then becomes largely inapplicable. However, they state that quick and reliable delivery is possible also in regulated environments, although accomplishing this might not be as straightforward as with simpler environments and delivery models. For instance, continuous integration can be more problematic for embedded software, since it is not always

possible to build and test on the target hardware but instead simulation is required. Furthermore, continuous deployment for embedded systems requires interconnected devices and IoT architecture, where the need for continuous functional safety and security must also be taken into consideration.

The main challenge in using plan-driven methods for medical device software development is in requirements management, particularly in the inability to accommodate changes once development has begun [14]. This is an increasing problem due to the growing size and complexity of software in medical devices. Agile methods for medical device software development have been suggested to overcome these issues [14]. In this paper we examine the role DevOps could have in medical device software development.

IV. DEVOPS AND MEDICAL SOFTWARE DEVELOPMENT STANDARDS

We examined two medical device and health software IEC/ISO standards, which were suggested by an industrial partner working on medical device software. The standards are *IEC 62304 - Medical Device Software - Software Life Cycle Processes* [6] and *IEC 82304-1 - Health Software - Part 1: General Requirements for Product Safety* [7]. These standards are related, as IEC 82304 uses the life-cycle model of IEC 62304 while giving eases in verification activities. The intended medical use of the end product regulates, which one of these standards is applicable for the software under development. IEC 62304 is for any medical device that contains hardware provided by the device manufacturer whereas IEC 82304-1 is for health software that runs on general purpose hardware that may be acquired and controlled by the customer. In addition, IEC 62304 is also applicable for medical software used for medical treatment or diagnosis, even if the software runs on general purpose hardware.

IEC 62304 defines a software life-cycle model that consists of planning, requirement gathering, design, implementation, verification, integration testing, system testing, and releasing activities as separate consequent phases. In addition, risk management, configuration management, and problem tracing are expanded over the whole life-cycle. In IEC 62304, the configuration management means tracing all the phases of all the identifiable deliverable of the software, such as the software units, documentations, test cases, and test reports, for example. Risk management should follow ISO 14971 risk management for medical devices standard [15]. Furthermore, ISO 13485 [16] – an extended version of ISO 9001 quality management standard for medical devices – should be used for quality management.

The software life-cycle model of IEC 62304 is adopted by IEC 82304-1 as well. However, IEC 82304-1 adds additional processes and activities before and after the life-cycle model to be complete standard to satisfy all the requirements of regulatory authorities. Such additional parts include user instructions' contents and post-market activities, for instance. It also provides simplifications on verification tasks, as it is not applicable for life threatening medical devices.

Neither of the standards specifies the practical execution of these processes and activities. Thus it is possible to use any tools and waterfall or agile process as long as the required activities are completed in accordance to the standards.

IEC 62304 describes threat classes A-C that vary on the threat level to patient from non-injury (Class A) to life threatening (Class C). These classes are subsets in such manner that all the clauses in the standard are in effect for Class C, whereas Class A is relieved from the most demanding verification tasks. This study was made concentrating on Class C medical devices as it covers the whole standard.

Table I summarizes our findings of how these two standards relate to DevOps. The findings are elaborated in the following sections. For IEC 62304, we discuss traceability and problem resolution process in detail, as we realized possible approaches for these from existing DevOps and work-flow tools during our analysis. The presented obstacles and benefits are the most observable ones that can be distinguished without exercising the standards in practice. Thus, there may be additional ones that are not discussed in this paper. Finding these are left for future work.

A. IEC 62304 specifics for DevOps approach

Based on our study, we found that the following clauses of IEC 62304 are notable for deploying DevOps practices:

- Clause 5.5.3 requires that all software units meet verification acceptance criteria before entering integration activities and integration testing. Thus, in pedantic execution, unit tests must be completed before integration testing may begin. This is incompatible with DevOps unless continuous integration takes place only after all units have passed unit testing.
- Clause 5.8.5 requires that the procedure and the environment of software creation must be documented. The procedure can be documented more precisely through integration in development tools, such as JIRA, GIT, and Jenkins, for instance.
- Clause 5.8.6 requires all tasks and activities to be completed before the software is released. This is the most significant obstacle for continuous deployment of DevOps, as any pending task – such as unfinished documentation – will halt the delivery in pedantic execution, and effectively eliminate the benefits and use of continuous deployment in the end product. Also, regulatory authorities have requirements for post-market updating, and the manufacturer is typically eligible to show compliance again before releasing the update. For example, IEC 62304 describes a software maintenance life-cycle for these purposes, which in practice follows the software development life-cycle. Obviously, during the development life-cycle, the manufacturer may use internal continuous deployment for testing purposes.
- Clause 5.8.8 requires that the software release must be repeatable. DevOps can help to fulfill this clause, as automated environment deployment tools, such as Docker, allow using invariable containers, which ensure that the installation can be repeated.
- Clause 8. Requires management of configuration items, such as providing unique identifiers and version control for all items. DevOps tools typically provide traceability for items represented in the tool, for example, test units and their runs can be traced directly to the software unit and its version. However, DevOps tools could provide more by allowing compliant tracing starting from the documentation of the requirements. This would require sharing the identifiers used in the tools.

In addition to the above clauses, IEC 62304 requires that all configuration items must be traceable to system requirements directly, or indirectly through other configuration items. Traceability is described explicitly only between system requirements, software requirements, software system test, and risk control (Clause 5.1.1 in IEC 62304). However, the risk control requires traceability of hazards and risk control measures to software items (Clause 3.5 in ISO 14971 and Clause 7.3.3 in IEC 62304), which in practice requires direct or indirect links between software items, verification items, and system requirements. The standards basically require that the traceability is constantly kept up-to-date, if any requirement or item changes.

DevOps can help to ensure tracing requirements by providing identifiers and link chains to connect software units between requirements, test cases, and test reports. For example, we found out that GIT, Jenkins (<https://jenkins.io/>), and JIRA (<https://jira.atlassian.com/>) can be connected if requirements are presented as issues in JIRA, but setting this up and using it is not straightforward nor obvious for the users. If implemented, existing links in JIRA issues and respective items in Jenkins and GIT allow tracing of involved configuration items, for example, in the case of changing system requirement.

Another notable aspect of IEC 62304 is the problem resolution (Clause 9 in IEC 62304), which requires that all the problems identified during the life-cycle must follow a process that ensures correct actions for each problem anomaly, such as completion of documentation and necessary corrective tasks. DevOps tools could auto-generate these anomalies when one is found in continuous integration. Also, the personnel should add the anomalies they find in documentation into the same tracing system.

We identified that JIRA can be used to follow all configuration items in the life-cycle by implementing different configuration items as separate issues with separate tailored JIRA work-flows. This way a new issue will be created and linked in a case of a problem, which would ensure all problems to be documented and resolved according to the required problem resolution process.

A continuous integration tool could function together with work-flow software, to automate the move of traced software items between verification activities of the life-cycle during completion of unit and integration tests. We found out that with JIRA and Jenkins, the work-flow would still require personnel to be aware of the life-cycle, because each step cannot be automatized with enough visibility and instructions for unaware personnel, for example, to fulfill autogenerated

TABLE I
SUMMARY OF THE MOST OBSERVABLE CLAUSES IN THE STANDARDS AFFECTING DEVOPS TECHNIQUES

Standard	Key point	DevOps techniques	Benefit or Obstacle
IEC 62304 - Clause 5.5.3	Software unit verification	Continuous integration	Obstacle
IEC 62304 - Clause 5.8.5	Software development procedure	Development tools	Benefit
IEC 62304 - Clause 5.8.6	Software release	Continuous deployment	Obstacle
IEC 62304 - Clause 5.8.6	Deployment repeatability	Deployment tools	Benefit
IEC 62304 - Clause 8	Item identification	Development tools	Benefit
IEC 82304-1 - Clause 8.4	Post-market reporting	Deployment tools	Benefit
IEC 82304-1 - Clause 8.4	Updating responsibility	Continuous deployment	Obstacle

problem resolution. Possibilities to extend JIRA with scripts was not studied. Thus, this might be solved with additional research and development.

B. IEC 82304-1 specifics for DevOps approach

IEC 82304-1 describes additional processes for post-market activities (Clause 8), which are typically required by regulatory authorities. Reporting detected errors is presented as an optional task in Clause 8.4 prior to deploying updated software. For example, FDA requires manufacturer to submit periodic reports that list incidents of any malfunctions or peculiarities on the use of the medical device that could have led to a death or serious injury. We argue that DevOps combined with IoT technologies could help detecting, collecting, and reporting these misuses by automatically collecting usage data of the device for the manufacturer. For example, the device could halt operations until the user submits an error report of the emergency shutdown (assuming the halting does not propose any risks to patients).

As IEC 82304-1 uses the life-cycle of IEC 62304, the clauses of IEC 62304 concern it as well. Additionally, we found that continuous deployment through remote updating to customer is not possible with IEC 82304-1 due to the requirement of transferring responsibility to the customer of taking the update into use (Clause 8.4).

V. SUMMARY AND CONCLUSIONS

Adopting modern software development approaches in regulatory environments is not simple. In this paper, we have discussed adopting DevOps methods in tightly regulated software development of medical devices. Two IEC standards, IEC 62304 and IEC 82304-1, were examined for clauses that either conflict with or benefit from DevOps type of an approach. The main findings were that the standards require special attention from continuous integration and prevent using continuous deployment after the deployment to the customer. Therefore, new tools and methods should be specifically developed for using DevOps in regulated software development.

Main future work will be developing such DevOps methods, which could help medical device software development to deal with the requirements of pedantic tracing, documentation, repeatability, deployment, maintenance, and post-market activities of the development process. We expect the key points to be creating links between all the items,

tasks, and activities of the process to automate traceability, verification, and other tedious yet necessary tasks, such as completing the problem anomalies for the problem resolution process. Creating methods that are applicable across multiple standards may be difficult, but close discussion should be established between the regulatory authorities, standardization, and developers over the needs of software development for future editions of the standards. This will require experiments to pinpoint crucial tasks of the standards that cannot be eased in favor of DevOps. Scientific community could help in this experimenting with further analysis of medical device software and DevOps.

REFERENCES

- [1] K. Schwaber, "Scrum development process," in *Business Object Design and Implementation*. Springer, 1997, pp. 117–134.
- [2] K. Beck, *Test-driven development: by example*. Addison-Wesley Professional, 2003.
- [3] —, *Extreme programming explained: embrace change*. Addison-Wesley Professional, 2000.
- [4] O. Cawley, X. Wang, and I. Richardson, *Lean/Agile Software Development Methodologies in Regulated Environments – State of the Art*. Springer Berlin Heidelberg, 2010, ch. Proc. Lean Enterprise Software and Systems: First International Conference, LESS 2010., pp. 31–36.
- [5] P. Debois, "DevOps: A software revolution in the making," *Journal of Information Technology Management*, vol. 24, no. 8, pp. 3–39, 2011.
- [6] IEC, "62304: 2006 medical device software – software life cycle processes," *International Electrotechnical Commission, Geneva, Switzerland*, 2006.
- [7] —, "82304-1: Health software – part 1: General requirements for product safety," *International Electrotechnical Commission, Geneva, Switzerland*. pp. 30., 2012.
- [8] M. Poppendieck and T. Poppendieck, *Lean software development: an agile toolkit*. Addison-Wesley Professional, 2003.
- [9] M. Fowler, "ContinuousDelivery," <http://martinfowler.com/bliki/ContinuousDelivery.html>, Apr. 2013, accessed: 2015-10-21.
- [10] J. Smeds, K. Nybom, and I. Porres, "DevOps: a definition and perceived adoption impediments," in *International Conference on Agile Software Development*. Springer, 2015, pp. 166–177.
- [11] W. Royce, "Managing the development of large software systems," in *Proc. IEEE WESCON*, vol. 26, no. 8, 1970, pp. 328–338.
- [12] L. E. Lwakatere, T. Karvonen, T. Sauvola, P. Kuvaja, H. H. Olsson, J. Bosch, and M. Oivo, "Towards DevOps in the embedded systems domain: Why is it so hard?" in *System Sciences (HICSS), 2016 49th Hawaii International Conference on*. IEEE, 2016, pp. 5437–5446.
- [13] C. Ebert, G. Gallardo, J. Hernantes, and N. Serrano, "Devops," *IEEE Software*, vol. 33, no. 3, pp. 94–100, 2016.
- [14] M. Mc Hugh, O. Cawley, F. McCaffery, I. Richardson, and X. Wang, "An agile v-model for medical device software development to overcome the challenges with plan-driven software development lifecycles," in *Software Engineering in Health Care (SEHC), 2013 5th International Workshop on*. IEEE, 2013, pp. 12–19.
- [15] *Medical devices — Application of risk management to medical devices - ISO 14971:2007(E)*. IEC/ISO, 2007.
- [16] *Medical devices — Quality management systems — Requirements for regulatory purposes - ISO 13485:2003(E)*. ISO, 2003.